5

Linear and Logistic Regression

CONTENTS

Linear regression is the most demonstrated representation of how an ML algorithm finds loss with the ground truth and learns a decision boundary or regression line depending on whether the task is classification or regression respectively. In section 1.5.2 we will discuss the linear regression algorithm used for regression tasks. It learns a regression line by fitting different combinations of slope and y-intercept of the line and choosing the one with the least loss. The process of learning a curve is not limited to regression task, in section 6.1 we will discuss logistic regression, an extension of curve learning for classification. Since the classification task has classes and most samples are aligned with levels in the classification dataset. It turns out the straight line as section 1.5.2 is not a good curve to work with a classification data set. Section 6.1 uses another non-linear curve known as sigmoid, however, the process of fitting the curve and choosing the curve with the least loss is oblivious to the curve chosen. Without loss of generality, it could be said any curve shape could be fitted on data, and this curve can act as a linear regression line or decision boundary for classification.
The process of curve fitting is supposed to learn the general behaviour of data which should be specific to the dataset. ML expects the algorithm to work for

any unseen data, if the curve-fitting process does not learn general behaviour it is said to be overfitting the data. The latter part of the chapter in the section 5.3 discusses different regularization to reduce overfitting of data.

5.1 Linear Regression and Gradient Descent

As described in section 1.5.2, regression analysis is one of the simplistic supervised learning algorithms that is used to determine a relationship between the dependent also known as the target variable, and independent variables. Such analysis is used for time series analysis, forecasting applications, and predictive analysis. The relationship between the target and independent could be linear as well as nonlinear, for both cases the regression analysis would be implemented in different ways. The regression analysis involves fitting the best possible line among many possible lines in the given set of points. For a linear set of points, the exact fit is straightforward, however, for a nonlinear set of points the line is fit according to the best possible fit since an exact fit is not available. The best possible fit involves minimizing the error function of the distance between points in the dataset and on the candidate regression line. The set of points can be continuous or discrete, they may follow different distributions. Based on these variations, different regression techniques could be broadly divided into Logistic and Linear regression.

5.1.1 Linear Regression and the loss function

Consider we have four points as shown in figure 5.1. Suppose we have to fit a straight line that passes through all five points. For humans, this is a trivial task but for machines, this task is not as simple as for humans. The machine has to try a different combination of lines (sweeping the line) and find the difference also known as a loss between the line under the trail and the given set of points. The loss becomes less as the line under trail comes near the given set of points and gets higher as the line under trail moves away from the given set of points. Let's consider that the y-intercept of the line is constant and we only change the slope. So the steps for this procedure are as follows.

- Take a line with any random slope θ_1.
- Change the slope in each step by adding step size to the slope, shown in figure 5.2.
- For each step compute the loss function between actual points and the line being swept.

The loss is the difference between the actual set of points and the points taken on the line swept. The major goal is to find the line which is the nearest fit to

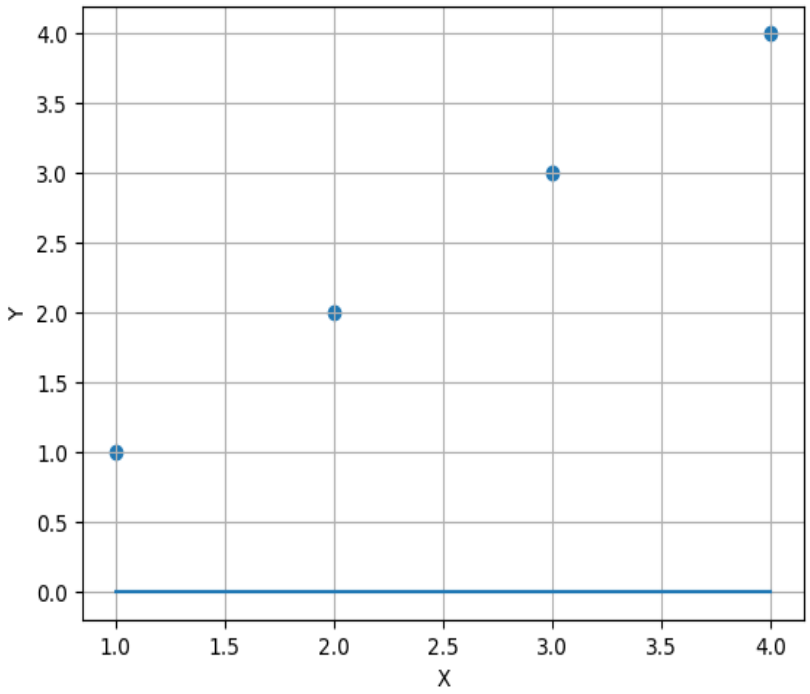

FIGURE 5.1
Given a set of points to fit regression on

the given set of points. The loss function can be given as in equation 5.1.

$$L = J(\theta_0, \theta_1) = \frac{1}{N}\sum_{i=1}^{N}(y - \hat{y})^2 \tag{5.1}$$

It could be noticed that since the loss function has a square in the difference the line at an equal distance below or above the actual set of points would give the same loss. In the given example since the set of points is linear, there is an exact possible fit and zero loss. However, in the case of a non-linear set of points, there will not be an exact fit and the effort would be to fit the best line in the least square sense or terms of minimum mean square error as shown in equation 5.1. In this example we have swept the slope of the line m or θ_0, however, we can also sweep the y-intercept m or θ_1 by keeping the slope constant. In both cases, the loss function shown in figure 5.3 represents the difference between an actual set of points and the line being swept. The loss function is parabolic as obvious from equation 5.1 which is $y = x^2$. In figure

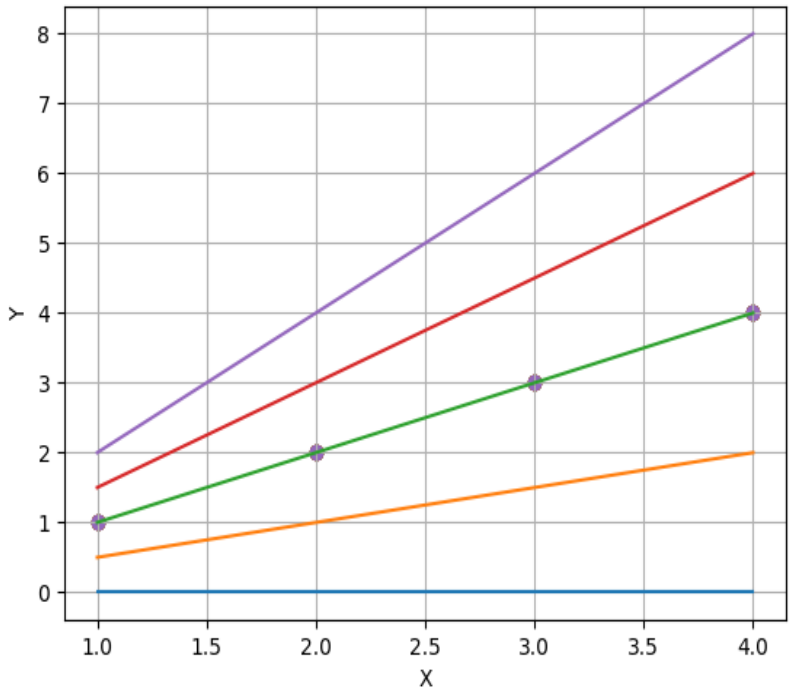

FIGURE 5.2
Sweeping the slope of the line over a given set of points

5.3 it's not a perfect parabola since the sweep steps are very less, however, with a smaller step size it approaches a perfect parabola. It is conventional to use θ_0 instead of m and θ_1 instead of c, since θ_i is used for parameters to be estimated. For the given loss function in figure 5.3, the major goal is to determine the minima since it represents the best-fit line with minimum loss. To achieve this, section 5.1.2 elaborates a systematic algorithm known as gradient descent.

```
import numpy as np
import matplotlib.pyplot as pt

x=np.array([1,2,3,4])
y=np.array([1,2,3,4])

theta1 = 0 #Keep the slope constant
x_line = np.array([1,2,3,4])

m_list = [0,0.5,1,1.5,2]
```

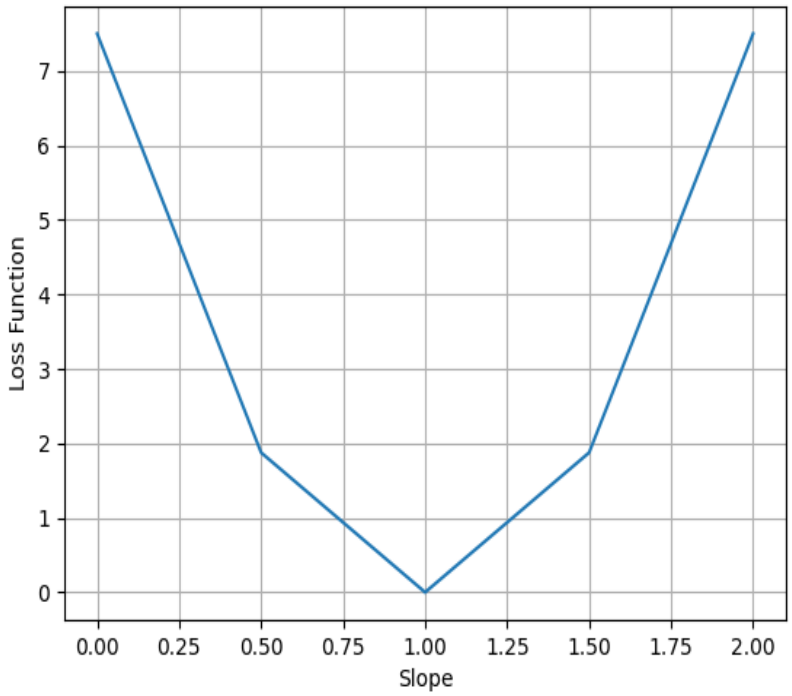

FIGURE 5.3
The loss function

```
loss=[]
i=0
for m in m_list:
# Calculate corresponding y values
  theta0=m
  y_predict = theta0*x_line + theta1
  y_line=x*1+0
  loss.append(np.mean(((y_line - y_predict)**2)))
  pt.plot(x_line, y_predict, label=f'm = {m}')
pt.scatter(x,y)
pt.xlabel('X')
pt.ylabel('Y')
pt.grid()
pt.show()
```

Code Listing 5.1
Sweeping the slope of the line

The Python code in Listing 2.1 shows how to sweep a line and get its loss

function using Python. Line 10 takes the set of slopes while line 13 iterates through them taking one slope at a time. Based on that line 15 calculates the line to be swiped while line 16 is the given set of points. In line 17, the loss function is calculated using mean square error.

5.1.2 The Gradient Descent

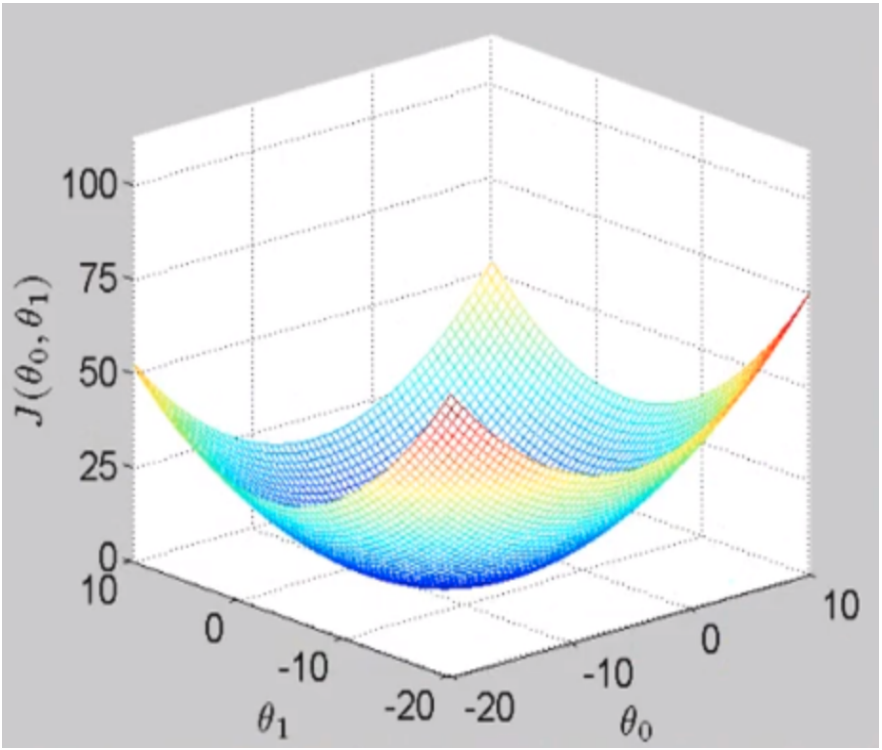

FIGURE 5.4
Three Dimensional graphs of two variables loss function

Gradient descent is an optimization algorithm commonly used in machine learning and other numerical optimization problems. The goal of gradient descent is to minimize a cost or loss function by iteratively adjusting the parameters of a model. The cost function measures the difference between the predicted and actual values of the dependent variable in a supervised learning problem. The basic idea of gradient descent is to compute the gradient of the cost function with respect to the parameters and update the parameters in the opposite direction of the gradient to minimize the cost function. In other words, the algorithm follows the direction of the steepest descent in the cost function to reach the minimum. The gradient is a vector that points in

the direction of the greatest rate of change of the function. In the case of a cost function, the gradient indicates the direction of the steepest increase in the cost. By taking the negative of the gradient, we get the direction of the steepest decrease in the cost. Figure 5.3 shows the loss function, where each point in this function corresponds to a certain line. In order to find the best-fit line in the given dataset, the minima of the loss function must be determined. Gradient descent (GD) discovers the minima in the given loss function using systematic iteration on the loss function. In the first step, any random point on the loss function is selected, and the GD goes step by step down the hill until it reaches the minimum on the curve.

1. Take the random value of an initial point on the loss function.
2. Repeat Step-3 to Step-4 until algorithm converges.
3. P_N is the new point, calculated from the old point, P_O. $P_N = P_0 - \eta\delta$, where η is step size or learning rate, δ is update.
4. Calculate δ by taking the derivative of the slope.

The loss function in equation 5.1 becomes equation 5.3 if the value of $\hat{y}$ is replaced by $\hat{y} = \theta_0 x + \theta_1$, where the predicted value $\hat{y}$ can also be written in terms of $h_\theta(x)$ meaning that $\hat{y}$ is a polynomial of θ, $\hat{y} = h_\theta(x) = \theta_0 x + \theta_1$, as shown in equation 5.2.

$$J(\theta_0, \theta_1) = \frac{1}{N}\sum_{i=1}^{N}(y_i - h_\theta(x))^2 \tag{5.2}$$

$$J(\theta_0, \theta_1) = \frac{1}{N}\sum_{i=1}^{N}(y_i - \theta_0 x - \theta_1)^2 \tag{5.3}$$

To find an update of each step two derivatives are calculated concerning θ_0 and θ_1, the derivative concerning θ_0 is given in equation 5.4 and for θ_1 is given in equation 5.5.

$$\frac{\partial J(\theta_0, \theta_1)}{\partial \theta_0} = (-2/N)\sum_{i=1}^{N}(x)(y - \theta_0 x - \theta_1) \tag{5.4}$$

$$\frac{\partial J(\theta_0, \theta_1)}{\partial \theta_1} = (-2/N)\sum_{i=1}^{N}(y - \theta_0 x - \theta_1) \tag{5.5}$$

The new point P_N is calculated by adding an update δ in the old point P_0, since the update is in the downward direction a negative sign is added with the update $P_N = P_0 - \eta\delta$. The η is called the learning rate or step size, which decides how long a single step would be. The updated equation for θ_0 and θ_1 are given in equation 5.6 and equation 5.10. The $\partial\theta_0$ and $\partial\theta_1$ are calculated using equation 5.4 and equation 5.5 respectively.

$$\theta_{0U} = \theta_0 - \eta \frac{\partial J(\theta_0, \theta_1)}{\partial \theta_0} \tag{5.6}$$

$$\theta_{1U} = \theta_1 - \eta \frac{\partial J(\theta_0, \theta_1)}{\partial \theta_1} \tag{5.7}$$

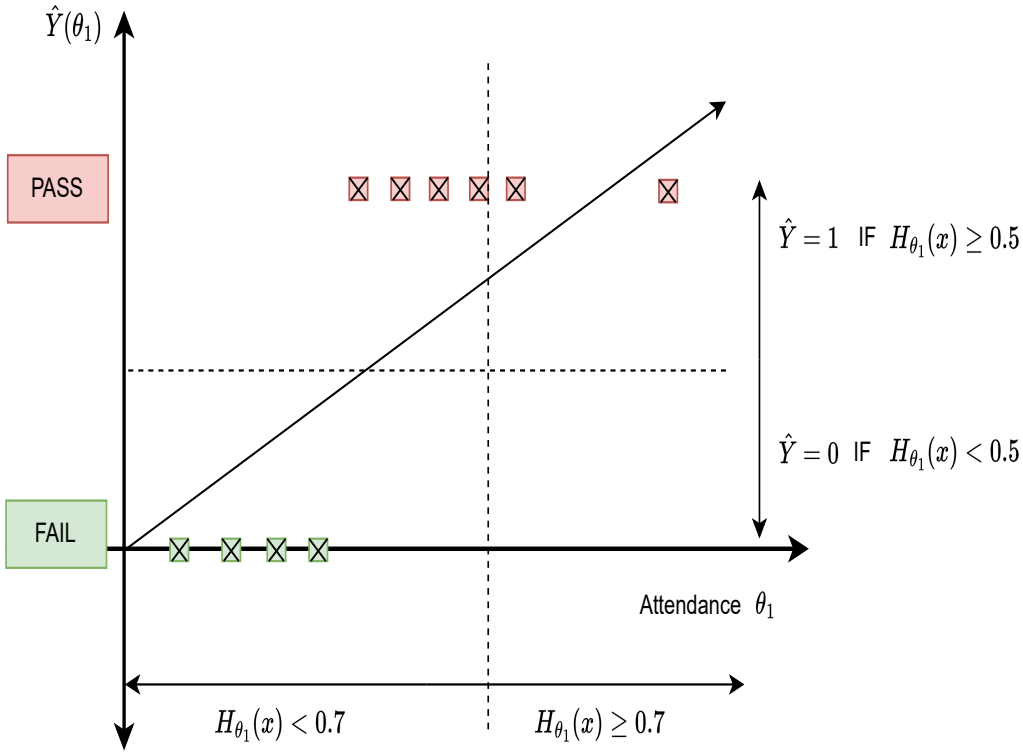

FIGURE 5.5
Regression on Classification data with an outlier

```
import numpy as np
def GradientDescent(X,Y):
  NIter=1000
  theta0_curr=0
  theta1_curr=0
  eta=0.01
  N=len(X)
  while NIter>0:
   yhat=  theta0_curr*X+theta1_curr
   updateTheta0=-(2/N)* sum(X*(Y-yhat))
   updateTheta1=-(2/N)* sum((Y-yhat))
```

```
    theta0_curr=theta0_curr-eta*updateTheta0
    theta1_curr = theta1_curr - eta * updateTheta1
    loss=(1/N)* sum((Y-yhat)**2)
    print("m{}, c{},loss{}, iterations{}".format(theta0_curr,
     theta1_curr,loss,NIter))
    NIter = NIter - 1

x=np.array([1,2,3,4,5])
y=np.array([5,7,9,11,13])
GradientDescent(x,y)
```

Code Listing 5.2
Gradient Descent in Python

The figure 5.4 shows the 3D graph of θ_0 and θ_1 and cost function $J(\theta_0, \theta_1)$. GD on this function means that from a random point on the curve

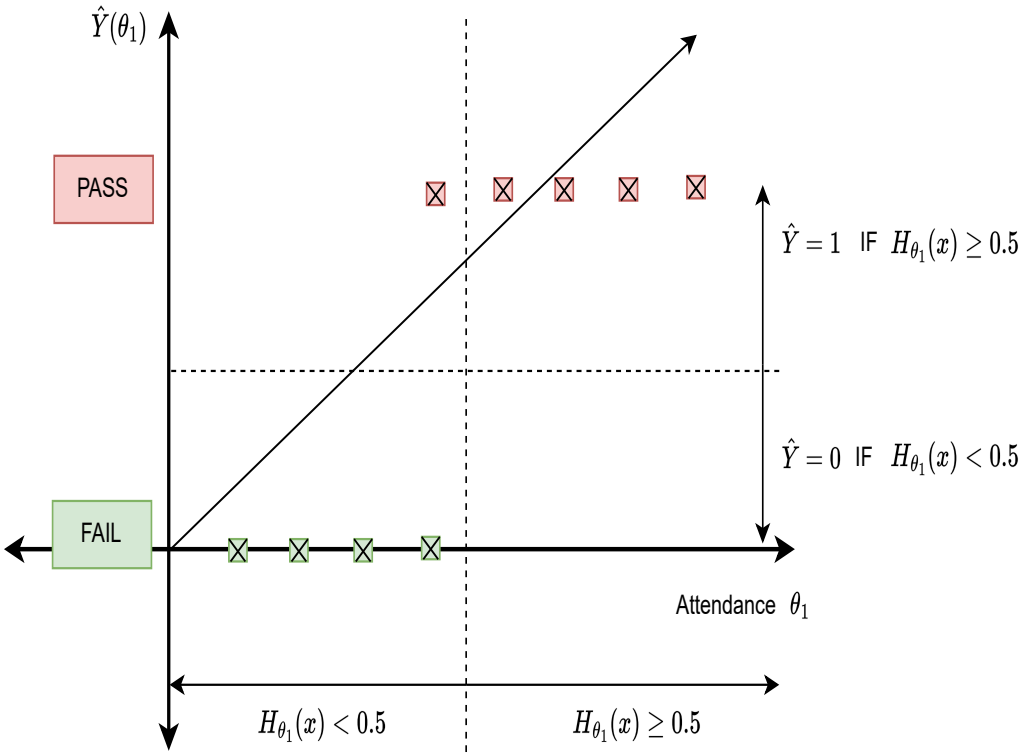

FIGURE 5.6
An Example of Regression on Classification data

5.1.2.1 The Learning Rate

Learning rate η is an important parameter that decides how large a step would be when going down the hill in gradient descent. If η is small the algorithm would take a very small step, which means the algorithm would converge very slowly. On the other hand, if η becomes too large the jumps would be very large and the algorithm would miss the minima on the loss function. Therefore, η is a learnable parameter that is tuned with hit and trail so that the algorithm is not too small and that convergence takes a long time. On the other hand, η is not so large that it misses the target.

5.1.2.2 Extension to Multifeature case

In the previous section, we dealt with the case where there were only two features θ_0 and θ_1. However, in particular settings, there might be M features. Equation 5.8 shows $\hat{y}$ having M features.

$$\hat{Y} = \sum_{j=1}^{M-1} \theta_j X_j + \theta_M \tag{5.8}$$

$$L = \frac{1}{N} \sum_{i=1}^{N} (y - \sum_{j=1}^{M-1} \theta_j X_j - \theta_M)^2 \tag{5.9}$$

$$\frac{\partial L}{\partial \theta_j} = \frac{-2}{N} \sum_{i=1}^{N} (X_j)(y - \sum_{j=1}^{M-1} \theta_j X_j - \theta_M) \tag{5.10}$$

5.2 Logistic regression

Linear regression has been discussed in the section 5.1.1, it can successfully learn models using data and predict values based on lines learned from data. However, linear regression is not suitable for classification because it is designed for predicting continuous numerical values, not categorical or binary outcomes. In classification problems, the outcome variable is usually categorical or binary, meaning it takes on a limited number of values (e.g., 0 or 1). Therefore, linear regression is not suitable for classification for two reasons. Firstly, linear regression cannot directly predict the probability of a binary outcome, since it may produce predictions outside the range of 0 to 1, which are not meaningful for binary classification. Secondly, a single outlier may shift the boundary condition which results in a reduction in the accuracy of the entire system. In summary, linear regression is not suitable for solving classification problems and we need some other regression to treat $Y \in \{0, 1\}$.

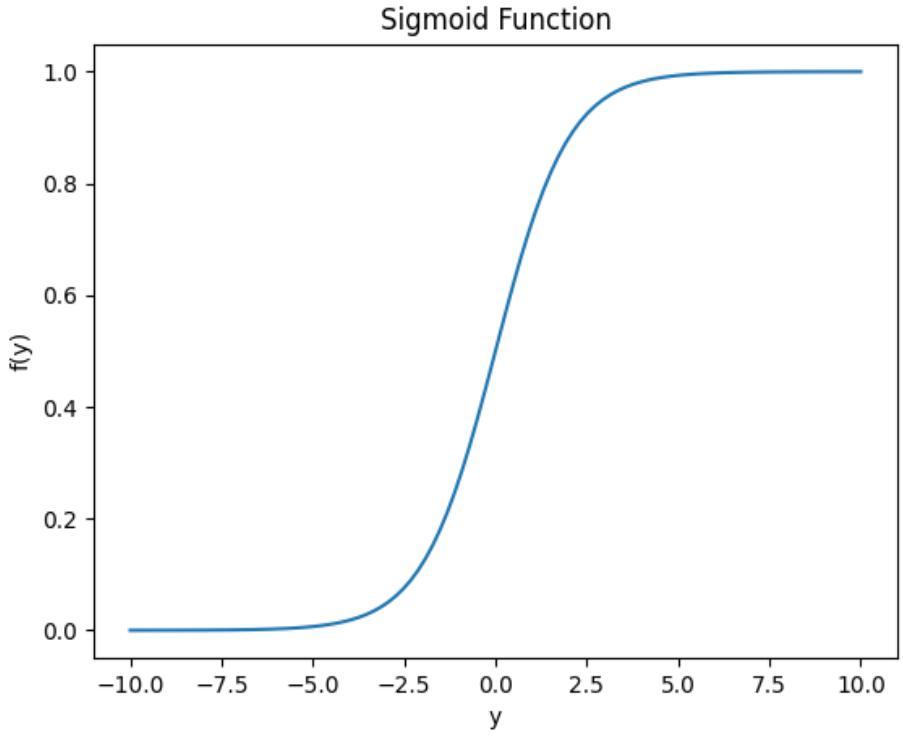

FIGURE 5.7
Sigmoid, the function for fitting in logistic regression

For example, deciding if the student will pass or fail the course based on his presence in the class. As a convention, we will treat 0 as a negative class and 1 as a positive class. In the given example, fail will be treated as a negative class while the pass will be treated as a positive class as shown in figure 5.6. The $\hat{Y}(\theta_1)$ has categorical labels i.e. pass and fail, so all samples will either lie on the x-axis or on the line along the pass. If we try to fit a straight line using this data as shown in 5.6 that could do classification in a way that if $H_{\theta_1(x)} \geq 0.5$ the predicted value $\hat{Y} = 1$, and if $H_{\theta_1(x)} \leq 0.5$ the predicted value is $\hat{Y} = 1$. However, the line learned in linear regression is strongly dependent on the data. For example, if there is a data sample that is an outlier and lies away from any of the classes, the line in linear regression will shift in an incorrect way as shown in figure 5.5. Besides, it could be observed that linear regression is a bad fit for Y-Axes with categorical labels, the $H_{\theta_1}(x)$ tends to go beyond 1 or less than 0. The output $\hat{y}$ might go beyond 1 or less than 0, we would like to have a function where $0 \geq H_{\theta_1}(x) \leq 1$.

In logistic regression instead of fitting a straight line, we need a curve that has two levels and a boundary that does not shift very easily with few data points. Also, it should not go beyond 1 and can not go less than 0 as shown in equation 5.11.

$$0 \leq h_\theta(x) \leq 1 \tag{5.11}$$

Figure 5.7 shows the Sigmoid function which has two levels and well defines the boundary between both classes. The mathematical representation of sigmoid is given in equation 5.12 and can also be written for $y = h_\sigma(x)$ as in equation 5.13.

$$f(Y) = \frac{1}{1 + e^{-Y}} \tag{5.12}$$

$$f(h_\theta(x)) = \frac{1}{1 + e^{-\theta^T X}} \tag{5.13}$$

Another interpretation of logistic regression besides being curve fitting and boundary condition is that it provides probabilities of dependent variables belonging to a particular class. For example, $H_{\theta_1}(X) \geq 0.8$ tells that the student has an 80% chance of passing the course. The output value of the hypothesis $H_{\theta_1}(X)$ tells the probability or chances of belonging to a positive or negative class. The probability of the dependent variable being positive is given as $P(Y = 1|X;\theta)$ while for negative is given as $P(Y = 0|X;\theta)$. The total probability can be therefore given as in equation 5.14.

$$P(Y = 0|X;\theta) + P(Y = 1|X;\theta) = 1 \tag{5.14}$$

Or by taking $P(Y = 0|X;\theta)$ on the other side can be written as in equation 5.15.

$$P(Y = 0|X;\theta) = 1 - P(Y = 1|X;\theta) \tag{5.15}$$

The entire discussion can be summarized as follows:

$$H_{\theta_1}(X) = P(Y = 1|X;\theta) = f(\theta^T X) \tag{5.16}$$

Where value of $f(h_\sigma(x)$ is given in equation 5.13. Each $\hat{y}$ can be understood from the boundary condition where $\hat{Y} = 1$ if $H_{\theta_1}(x) \geq 0.5$, similarly, $\hat{Y} = 0$ if $H_{\theta_1}(x) < 0.5$. By looking at figure 5.7 we can say that if $Y > 0$ the function $f(y) \geq 0.5$ or in other words $\theta^T X \geq 0.5$. With a similar logic, we can say that if $Y < 0$ $\theta^T X < 0.5$.
Generally, the regression has more than one feature as given in figure 5.17.

$$Y = \theta_1 X_1 + \theta_2 X_2 + \ldots \theta_M X_M \tag{5.17}$$

The equation 5.18 and equation 5.19 provides extension to equation 5.13 for multifeature case.

$$f(y) = \frac{1}{1 + e^{-(\theta_1 X_1 + \theta_2 X_2 + ... \theta_M X_M)}} \tag{5.18}$$

$$f(y) = \frac{1}{1 + e^{-\sum_{i=1}^{M} \theta_i X_i}} \tag{5.19}$$

5.2.1 Loss for Logistic regression

Although Sigmoid is a very good fit for data with categorical Y-axes, however, it produces a non-convex loss function if loss function in equation 5.2. The gradient descent in section 5.1.2 would fail if the loss function is non-convex. The mean square error (MSE) in the equation 5.2 is suited for continuous variables since it treats all output decisions equally without taking into account the difference between output predictions. On the other hand, we need a loss function that can penalize models that are overconfident or underconfident in their predictions. In other words, it gives a higher penalty to models that predict a high probability of the wrong class label or a low probability of the correct class label. Perhaps the most suited function for this purpose is $-log(y)$ for positive class since it gives zero when approaching 1 and gives a very high penalty for values approaching 0. Similarly, $log(y)$ is suitable for negative class, it can penalize values approaching 1 and gives almost no penalty for values approaching 0 as given in equation 5.20.

$$J(h_\theta(x), y) = \begin{cases} -log(H_\theta(x)) & \text{if } Y = 1 \\ -log(1 - (H_\theta(x))) & \text{if } Y = 0 \end{cases} \tag{5.20}$$

The total loss function $J(h_\theta(x), y)$ can be given as the sum of positive and negative classes as in equation 5.21.

$$J(h_\theta(x), y) = -ylog(H_\theta(x)) - (1 - y)log(1 - H_\theta(x)) \tag{5.21}$$

The sum of all samples can therefore be given as in equation 5.22. Interestingly the derivative of the sigmoid function is similar to that for MSE. As proved in section 5.4.1 the derivative of the sigmoid function $\frac{\partial J}{\partial \theta}$ is given as equation 5.23.

$$J(\theta) = -\frac{1}{N} \sum_{i=1}^{N} [y_i log(h_\theta(x_i)) + (1 - y_i) log(1 - h_\theta(x_i))] \tag{5.22}$$

$$\frac{\partial h_\theta(x)}{\partial \theta} = X h_\theta(x)[1 - h_\theta(x)] \tag{5.23}$$

Putting the value of the derivative of a Sigmoid function in cross-loss entropy in equation 5.21 results in equation 5.24. The derivative of a Sigmoid function in equation 5.24 is comparable to the derivative of MSE in the equation 5.5 or its extension to the multivariable case in equation 5.10. Therefore,

it could be proved that finding the loss function or the best-fit sigmoid using gradient descent is similar to linear regression.

$$\frac{\partial J}{\partial \theta} = \frac{1}{N}\sum_{i=1}^{N}[(h_\theta(x) - y_i)X] \tag{5.24}$$

The Python code for logistic regression is given below.

```
import numpy as np
from sklearn.linear_model import LogisticRegression
import matplotlib.pyplot as pt


#features
x=np.arange(10).reshape(-1,1)
#true labels
y=np.array([0,0,0,0,1,1,1,1,1,1])

model=LogisticRegression()

model.fit(x,y)

prediction=model.predict([[4]])
print(prediction)


pt.scatter(x,y)
pt.xlabel('X')
pt.ylabel('Y')
pt.grid()
pt.show()
```

Code Listing 5.3
Python code for logistic regression

5.3 Regularization

The major role of the ML model is to learn generalization from the data. However, in some cases, the model would be very specific to the data and does not learn generalization such a situation is known as overfitting as further explained in section 5.3.1. Regularization is used to reduce overfitting in regression and build a model which is generalized. Overfitting occurs when the model fits specific to training data and performs poorly for test data, further discussed in sub-section 5.3.1. There are two types of regularization used in regression:

- L_1 regularization or Lasso regularization adds a penalty term to the loss function that is proportional to the absolute value of the model coefficients.

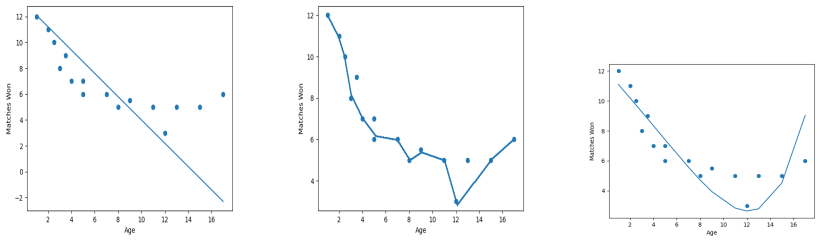

FIGURE 5.8
Over, Under and good fit

This leads to sparse models where some of the coefficients are exactly zero, effectively removing some of the features from the model further discussed in subsection 5.3.2.

- L_2 regularization or Ridge regularization adds a penalty term to the loss function that is proportional to the squared value of the model coefficients. This encourages the model to have small, non-zero coefficients for all features, further explained in subsection 5.3.3.

Both L_1 and L_2 regularization can be applied to linear regression, logistic regression, and other types of regression models. A hyperparameter controls the strength of the regularization, typically denoted as λ, which needs to be tuned to find the optimal balance between model complexity and generalization performance. The subsection will further explore the regularization, subsection 5.3.1 formally define over and under fitting.

5.3.1 Over and Under Fitting

Overfitting is a concept where a model works exceptionally well on training data but gives very low accuracy on testing data. It could be calculated by subtracting test accuracy from training accuracy. This effectively means that the data has a lot of variance and training of different instances of datasets gives varying accuracy. The very naive interpretation of straight line equation 5.25 and equation 5.26 is that during training the model m or θ_1 and c or θ_2 also known as weights are learned.

$$\hat{y} = mx + c \tag{5.25}$$

$$\hat{y} = \theta_1 x + \theta_2 \tag{5.26}$$

The first weight θ_1 tells how much input data X has the influence to

generate $\hat{y}$. If during training low value of θ_1 is learned that means X has a low influence also known as underfitting and if a high value of θ_1 is learned that means X has a very strong influence in generating $\hat{y}$. The left-most part of the figure 5.8 shows the underfit model, where the line has been learned but linear regression does not properly represent the given points. In the center, the model is overfitting since it has learned a polynomial that is too specific to the data, and the generalized trend of the data is not learned. Such an overfit curve will give very high accuracy on the training data, however, will perform very poorly on the testing dataset. The right-most curve of figure 5.8 is the best-suited curve since the polynomial learns the general trend of the data as well as is not specific to the data.

5.3.2 Least Absolute Shrinkage and Selection Operator (Lasso) Regression

In order to under Lasso we have to observe the figure 5.8, the left-most curve is $\hat{Y} = \theta_1 + \theta_2 X$, while the curve in the center and on the right is higher order polynomial as in equation 5.27.

$$\hat{Y} = \theta_1 + \theta_2 X + \theta_3 X^2 + \theta_4 X^3 + \theta_5 X^4 + \theta_6 X^5 \tag{5.27}$$

Since the overfit curve and best-fit curve are both higher-order polynomials, the only difference between overfitting and best fit is defined by values of $\theta_3 \dots \theta_6$. Also, that equation 5.27 can be reduced to a straight-line equation on the left-most by putting $\theta_3 \dots \theta_6$ equal to zero. Similarly, the difference between the polynomial in the center (overfitting) and on the right (best fitting) is the value of the polynomial coefficients $\sum_{k=2}^{6} \theta_k$. This effectively means that by subtracting or adding the values with θ_k.

5.3.3 Ridge Regression

“nobreak

5.4 Annexure

This section will provide proof for taking the derivative of the sigmoid function and putting the derivative in the cross-loss entropy function. Both are used in the logistic regression, section 6.1.

5.4.1 Derivative of $h_\theta(x)$

Consider a sigmoid in equation 5.28.

$$h_\theta(x) = \frac{1}{1 + e^{-\theta^T x}} \tag{5.28}$$

If we take the derivative of the Sigmoid function in equation 5.28, the result is shown in equation 5.29.

$$\frac{\partial h_\theta(x)}{\partial \theta} = \frac{\partial}{\partial \theta}(1 + e^{-\theta^T X})^{-1} \tag{5.29}$$

Taking the derivative of $(1+e^x)^{-1}$, as $-1(1+e^x)^{-2}$, the derivative of other than x is shown in equation 5.30.

$$\begin{aligned} \implies \frac{\partial h_\theta(x)}{\partial \theta} &= -(1 + e^{-\theta^T X})^{-2} \frac{\partial}{\partial \theta}(1 + e^{-\theta^T X}) \\ \implies \frac{\partial h_\theta(x)}{\partial \theta} &= -(1 + e^{-\theta^T X})^{-2} [e^{-\theta^T X} \frac{\partial}{\partial \theta}(-\theta^T X)] \end{aligned} \tag{5.30}$$

Taking the derivative of other than x the result is given in equation 5.31.

$$\begin{aligned} \implies \frac{\partial h_\theta(x)}{\partial \theta} &= -(1 + e^{-\theta^T X})^{-2} [e^{-\theta^T X} \frac{\partial}{\partial \theta}(-\theta^T X)] \\ \implies \frac{\partial h_\theta(x)}{\partial \theta} &= X e^{-\theta^T X} (1 + e^{-\theta^T X})^{-2} \end{aligned} \tag{5.31}$$

By applying mathematical tricks on equation 5.31 the result is given in equation 5.33 the derivative of $h_\theta(x)$ is given as $h_\theta(x) = X h_\theta(x)[1 - h_\theta(x)]$.

$$\begin{aligned} \implies \frac{\partial h_\theta(x)}{\partial \theta} &= \frac{X e^{-\theta^T X}}{(1 + e^{-\theta^T X})^2} \\ \implies \frac{\partial h_\theta(x)}{\partial \theta} &= X \frac{1}{(1 + e^{-\theta^T X})} \frac{e^{-\theta^T X}}{1 + e^{-\theta^T X}} \end{aligned} \tag{5.32}$$

$$\begin{aligned} \implies \frac{\partial h_\theta(x)}{\partial \theta} &= X \frac{1}{(1 + e^{-\theta^T X})} \frac{e^{-\theta^T X}}{1 + e^{-\theta^T X}} \\ \implies \frac{\partial h_\theta(x)}{\partial \theta} &= X \frac{1}{(1 + e^{-\theta^T X})} \frac{1 + e^{-\theta^T X} - 1}{1 + e^{-\theta^T X}} \\ \implies \frac{\partial h_\theta(x)}{\partial \theta} &= X \frac{1}{(1 + e^{-\theta^T X})} [1 - \frac{1}{1 + e^{-\theta^T X}}] \\ \implies \frac{\partial h_\theta(x)}{\partial \theta} &= X h_\theta(x)[1 - h_\theta(x)] \end{aligned} \tag{5.33}$$

5.4.2 Calculating Cross loss entropy using $h_\theta(x)$

$$J(\theta) = -\frac{1}{N}\sum_{i=1}^{N}[y_i log(h_\theta(x_i)) + (1-y_i)log(1-h_\theta(x_i))] \tag{5.34}$$

$$\begin{aligned}\frac{\partial J}{\partial \theta} &= -\frac{1}{N}\sum_{i=1}^{N}[y_i \frac{\partial}{\partial \theta} log(h_\theta(x_i)) + (1-y_i)\frac{\partial}{\partial \theta} log(1-h_\theta(x_i))] \\ \frac{\partial J}{\partial \theta} &= -\frac{1}{N}\sum_{i=1}^{N}[\frac{y_i}{h_\theta(x_i)}\frac{\partial}{\partial \theta} h_\theta(x_i) + \frac{1-y_i}{1-h_\theta(x_i)}\frac{\partial}{\partial \theta}(1-h_\theta(x_i))]\end{aligned} \tag{5.35}$$

$$\frac{\partial J}{\partial \theta} = -\frac{1}{N}\sum_{i=1}^{N}[\frac{y_i}{\cancel{h_\theta(x_i)}}(X\cancel{h_\theta(x_i)}[1-h_\theta(x_i)]) + \frac{1-y_i}{1-h_\theta(x_i)}\frac{\partial}{\partial \theta}(1-h_\theta(x_i))] \tag{5.36}$$

$$\frac{\partial J}{\partial \theta} = -\frac{1}{N}\sum_{i=1}^{N}[y_i X(1-h_\theta(x_i)) + \frac{1-y_i}{\cancel{1-h_\theta(x_i)}} - (Xh_\theta(x)\cancel{[1-h_\theta(x)]})] \tag{5.37}$$

$$\frac{\partial J}{\partial \theta} = -\frac{1}{N}\sum_{i=1}^{N}[y_i X - \cancel{y_i \theta^T h_\theta(x_i)} - Xh_\theta(x) + \cancel{\theta^T h_\theta(x_i) y_i}] \tag{5.38}$$

$$\begin{aligned}\frac{\partial J}{\partial \theta} &= -\frac{1}{N}\sum_{i=1}^{N}[X(y_i - h_\theta(x))] \\ \implies \frac{\partial J}{\partial \theta} &= \frac{1}{N}\sum_{i=1}^{N}[(h_\theta(x) - y_i)X]\end{aligned} \tag{5.39}$$